

Tiny GPU — Massively Parallel Compute Core in Verilog

Subject : VLSI Design & Embedded Systems

Name : Rutuja Ravindra Salunkhe

Roll No. : ES-T-16/JY-11204

Branch : ENTC

Semester : V

Guide : Vaibhav Sir



Smt. Kashibai Navale College of Engineering, Vadgaon

Declaration

I hereby declare that the minor project report entitled "RTL Design & Testbench Simulation using EDA Playground" submitted to the Department of Electronics & Telecommunication my own independent work. All Verilog hardware logic, testbench stimulus models, waveform captures, and written analysis presented in this document were created by me.

Where support, hardware tools (EDA Playground), or standard references have been used, they have been properly cited and acknowledged in accordance with academic integrity guidelines.

Date : 05/09/2026

Place : pune



Student Name : Rutuja Ravindra Salunkhe

Roll No. : ES-T-16/JY-11204

Department of Electronics and Telecommunication

Countersigned by Project Guide

I have evaluated the student's independent work and verified that the declaration made by the student is true to the best of my knowledge. This minor project report is submitted with my approval.

Signature of Mentor

Guide Name : Vaibhav Sir

CERTIFICATE

This is to certify that Rutuja Ravindra Salunkhe (ES-T-16/JY-11204) has successfully completed the project titled " Tiny GPU — Massively Parallel Compute Core in Verilog " under my guidance in partial fulfillment of the requirements for the award of B.E. .

To the best of my knowledge, this work is an authentic record of the candidate's own work carried out under my supervision.

- Date : 05/09/2026
- Place : Pune

Vaibhav Sir,

Project Guide

INTRODUCTION

In modern computing, Graphics Processing Units (GPUs) have evolved beyond traditional image rendering into powerful parallel accelerators for data-intensive tasks such as artificial intelligence, scientific computing, and signal processing. Unlike traditional Central Processing Units (CPUs) optimized for low-latency sequential execution, GPUs maximize throughput by executing hundreds of parallel execution threads simultaneously across multiple dedicated shader cores.

This project focuses on the implementation, functional verification, and logic synthesis of a Verilog HDL-based 4-Core GPU Architecture. Designed on a Single Instruction, Multiple Data (SIMD) paradigm, the system utilizes a centralized dispatch unit to manage instruction streams and coordinate work across four independent execution cores operating in parallel.

The primary objectives of this project are:

- **RTL Architecture Implementation:** To model the core arithmetic and dispatch logic of a multi-core GPU using Verilog HDL.
- **Functional Verification:** To validate cycle-accurate execution and multi-core synchronization using Icarus Verilog (iverilog) and visualize execution traces using web-based waveform analysis tools.
- **Logic Synthesis:** To analyze hardware resource consumption, gate distribution, and silicon footprint by mapping the design through the Yosys synthesis suite.

By completing this end-to-end design flow, this report demonstrates the functional accuracy, multi-core output alignment, and primitive cell footprint of the 4-core GPU design.

OBJECTIVES

The primary goal of this project is to implement, verify, and analyze a functional 4-core GPU design using an open-source VLSI toolchain. The key objectives are:

- **RTL Architecture Design** : Implement a multi-core SIMD GPU in Verilog HDL, featuring a centralized dispatch unit and four independent shader cores with dedicated Arithmetic Logic Units (ALUs).
- **Parallel Execution & Control** : Validate multi-threaded instruction scheduling and ensure cycle-accurate synchronization across all four cores during parallel computation tasks.
- **Functional Verification** : Execute comprehensive testbench simulations using Icarus Verilog (iverilog/vvp) to verify data integrity and signal transitions across clk, rst, core_done, and multi-bit output buses.
- **Waveform Trace Analysis** : Parse and visualize VCD trace files using web-based waveform viewers to inspect runtime state changes and output values (out0 through out3).
- **Logic Synthesis & Gate Mapping** : Synthesize the Verilog design using Yosys to map RTL constructs into primitive logic cells, flip-flops, and multiplexers.
- **Hardware Resource Assessment** : Analyze the total cell count, wire density, and gate distribution (such as \$_ANDNOT_\$, \$_XOR_\$, and \$_DFFE_PP_\$) to evaluate the overall hardware footprint.

SYSTEM ARCHITECTURE

The 4-core GPU architecture is designed around a Single Instruction, Multiple Data (SIMD) processing pipeline. The top-level design module (`gpu_top.v`) integrates a central instruction dispatch unit with four parallel execution shader cores (`shader_core.v`), allowing identical vector operations to execute simultaneously across multiple data streams.

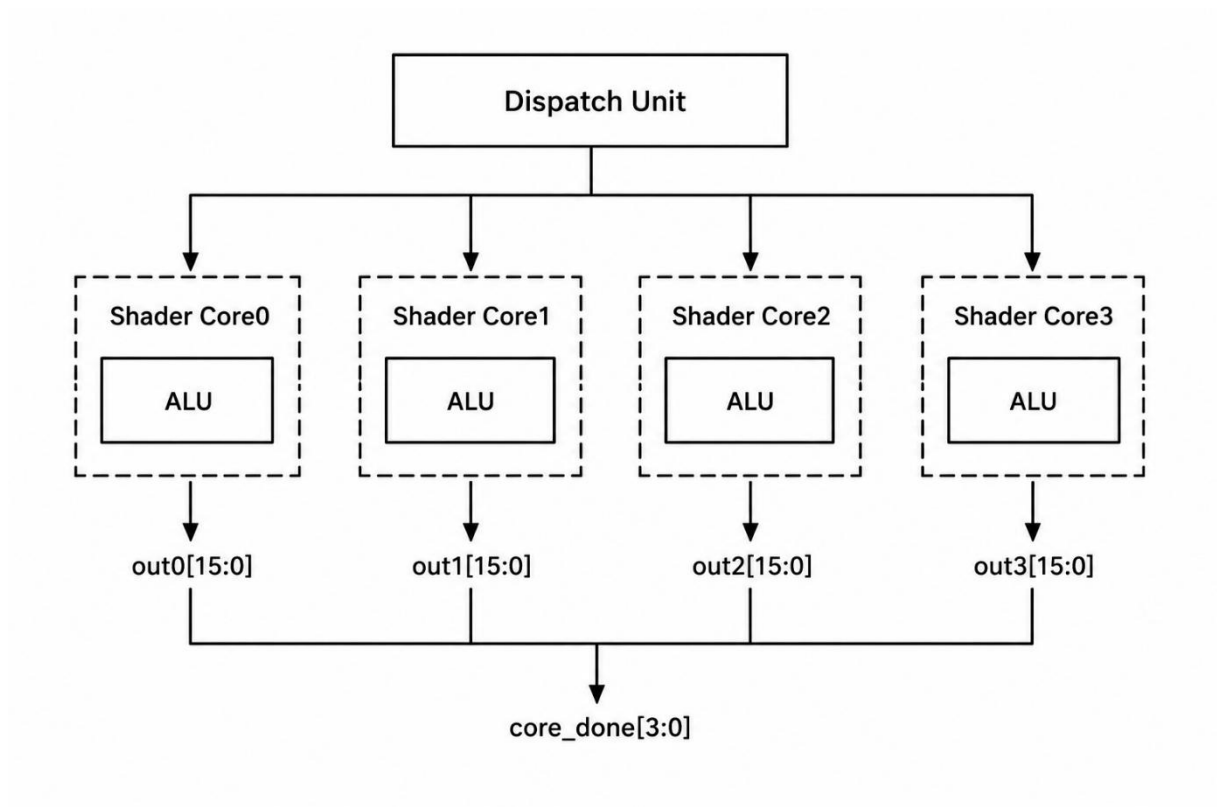


Fig. : Architectural Block Diagram

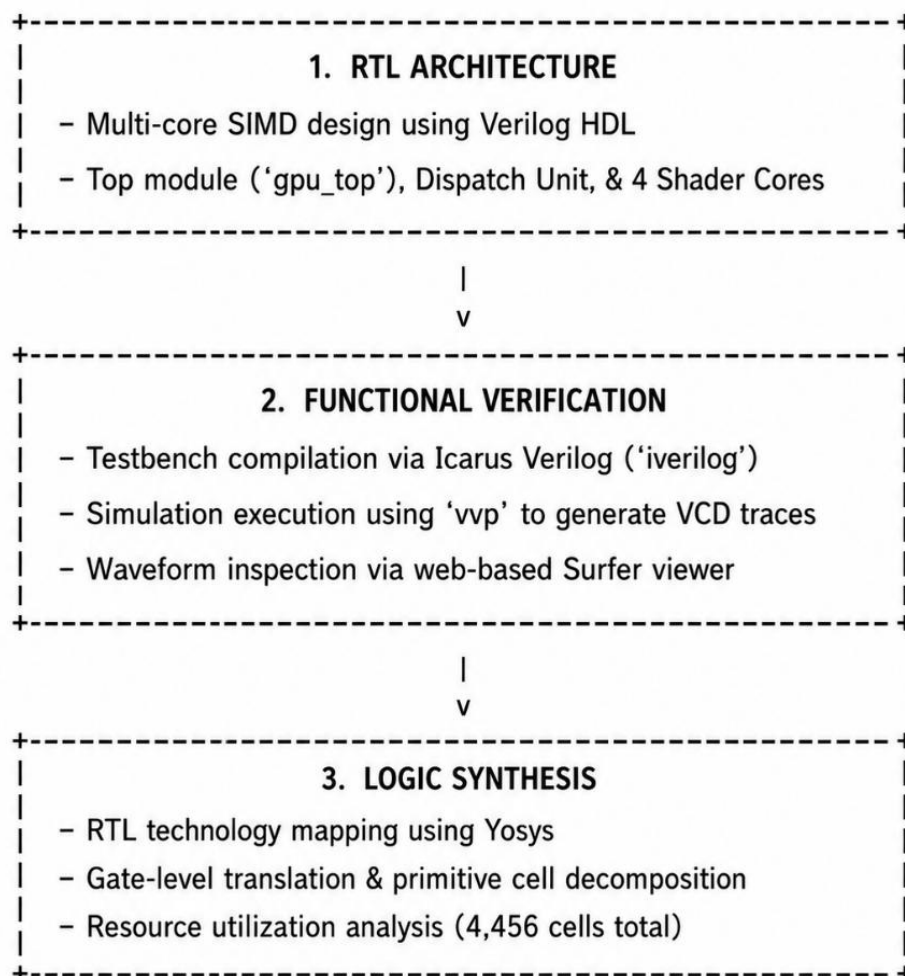
Subsystem Descriptions :

- Central Dispatch Unit (`dispatch_unit.v`) : Serves as the top-level control interface. It decodes incoming instruction blocks, broadcasts shared control signals, and manages task distribution across all instantiated execution cores.

- Shader Cores (shader_core.v) : Four identical execution cores (Shader Core0 through Shader Core3) operate in parallel. Each core processes its assigned vector dataset independently.
- Arithmetic Logic Unit (alu.v) : Instantiated within each shader core, the ALU executes core mathematical, bitwise, and logical vector operations.
- Memory & Control Interconnect (mem_ctrl.v) : Manages data transfers between local register files and external interface signals.
- Output & Handshake Signals :
 - out0[15:0] to out3[15:0] : Four independent 16-bit output data buses that drive computation results from each respective core.
 - core_done[3:0] : A 4-bit status flag vector providing core execution status and completion handshaking to the testbench.

METHODOLOGY AND VERIFICATION FLOW

The development and evaluation of the 4-core GPU project followed a structured front-end VLSI design flow using an open-source toolchain. The methodology is divided into three primary phases: architectural implementation, functional verification, and logic synthesis.



1. RTL Architecture & Design

- **Hardware Modeling:** Designed the top-level GPU module (gpu_top.v) and its underlying components using modular Verilog HDL.
- **SIMD Execution Model:** Implemented a centralized instruction dispatch unit to broadcast data and instructions across four parallel execution shader cores (shader_core.v), each containing a dedicated Arithmetic Logic Unit (alu.v).
- **Interconnect & Interface:** Configured multi-bit output buses (out0 through out3) and control signaling (core_done[3:0]) for multi-core synchronization.

2. Functional Verification Flow

- **Testbench Setup :** Developed tb/tb_gpu.v to supply clock, reset, and stimulus vectors to the unit under test.
- **Compilation & Simulation :** Compiled RTL sources using Icarus Verilog (iverilog) into a compiled executable binary (sim/gpu_sim) and executed it via the vvp runtime engine to capture console event logs.
- **Trace Analysis :** Exported value change dump (.vcd) files to analyze timing, cycle accuracy, and register state transitions using the Surfer waveform viewer.

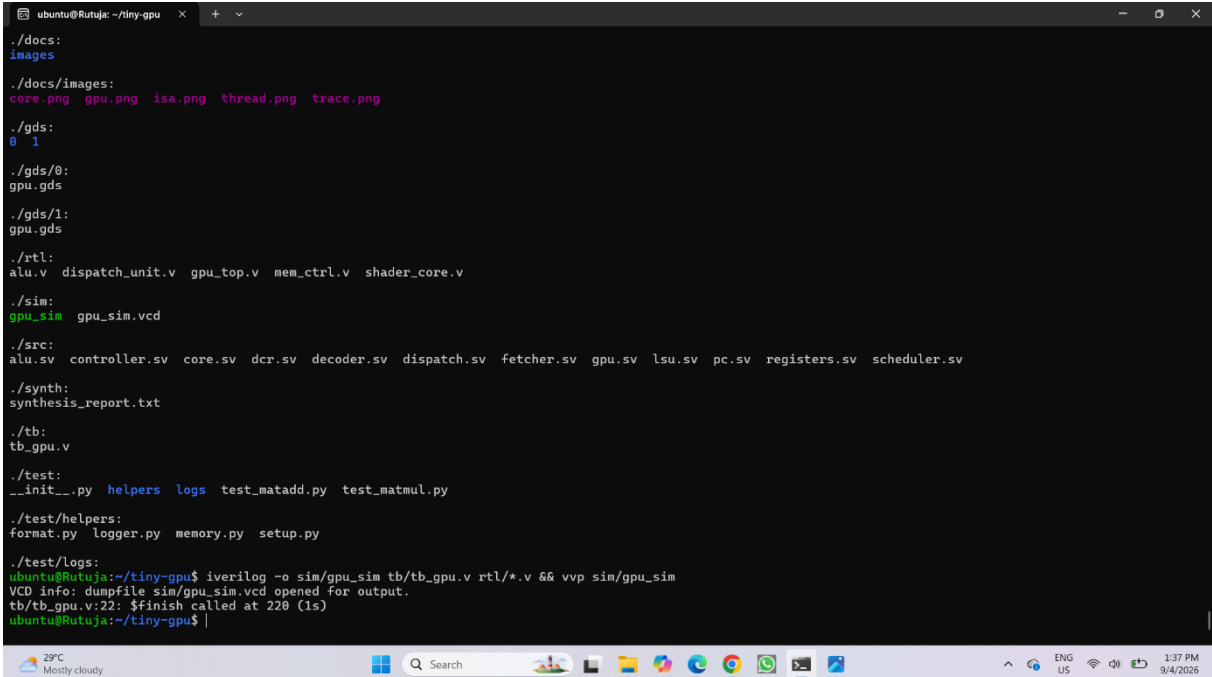
3. Logic Synthesis & Resource Analysis

- **Technology Mapping :** Executed automated logic synthesis scripts through Yosys to parse high-level RTL statements into gate-level netlists.
- **Cell Decomposition :** Translated arithmetic expressions and multiplexer trees into primitive standard cells (\$_ANDNOT_, \$_XOR_, \$_MUX_, etc.) and flip-flops (\$_DFFE_PP_).
- **Metric Extraction :** Extracted circuit properties, including design hierarchy depth, wire interconnect counts, and total hardware cell counts.

EXPERIMENTAL RESULTS

4.1 Functional Simulation Log

The testbench execution completed cleanly via Icarus Verilog without timing errors or assertion failures, terminating at time marker 220.

A terminal window titled 'ubuntu@Rutuja: ~/tiny-gpu' showing the execution of an Icarus Verilog simulation. The user enters commands to compile and simulate a GPU testbench. The log shows the compilation of various Verilog modules, the generation of a VCD file, and the successful completion of the simulation at time 220. The terminal output is as follows:

```
ubuntu@Rutuja: ~/tiny-gpu
./docs:
images
./docs/images:
core.png gpu.png isa.png thread.png trace.png
./gds:
0 1
./gds/0:
gpu.gds
./gds/1:
gpu.gds
./rtl:
alu.v dispatch_unit.v gpu_top.v mem_ctrl.v shader_core.v
./sim:
gpu_sim gpu_sim.vcd
./src:
alu.v controller.v core.v dcr.v decoder.v dispatch.v fetcher.v gpu.v lsu.v pc.v registers.v scheduler.v
./synth:
synthesis_report.txt
./tb:
tb_gpu.v
./test:
__init__.py helpers logs test_matadd.py test_matmul.py
./test/helpers:
format.py logger.py memory.py setup.py
./test/logs:
ubuntu@Rutuja:~/tiny-gpu$ iverilog -o sim/gpu_sim tb/tb_gpu.v rtl/*.v && vvp sim/gpu_sim
VCD info: dumpfile sim/gpu_sim.vcd opened for output.
tb/tb_gpu.v:220: $finish called at 220 (1s)
ubuntu@Rutuja:~/tiny-gpu$
```

Fig. : Terminal execution log displaying successful compilation and \$finish at time 220.

4.2 Waveform Verification

Waveform inspection confirms parallel execution across all four shader cores. Clock and reset transitions operate as expected, and values update across all 16-bit output buses (out0–out3) in alignment with the core_done complete flags.

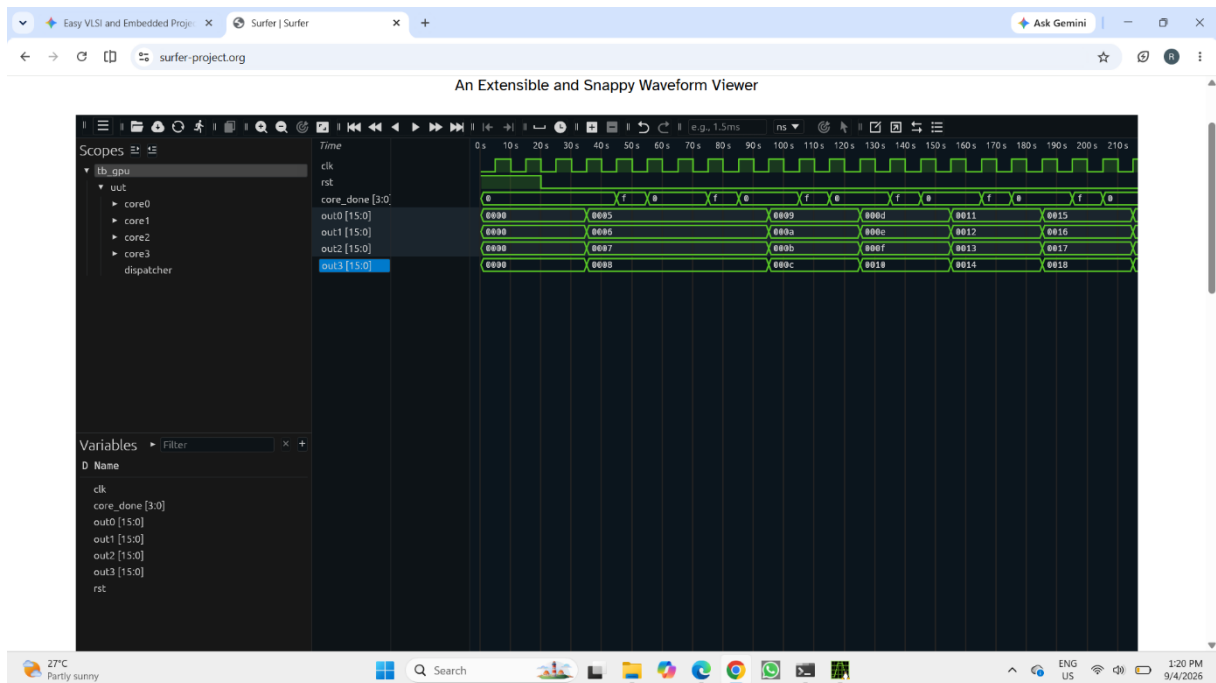
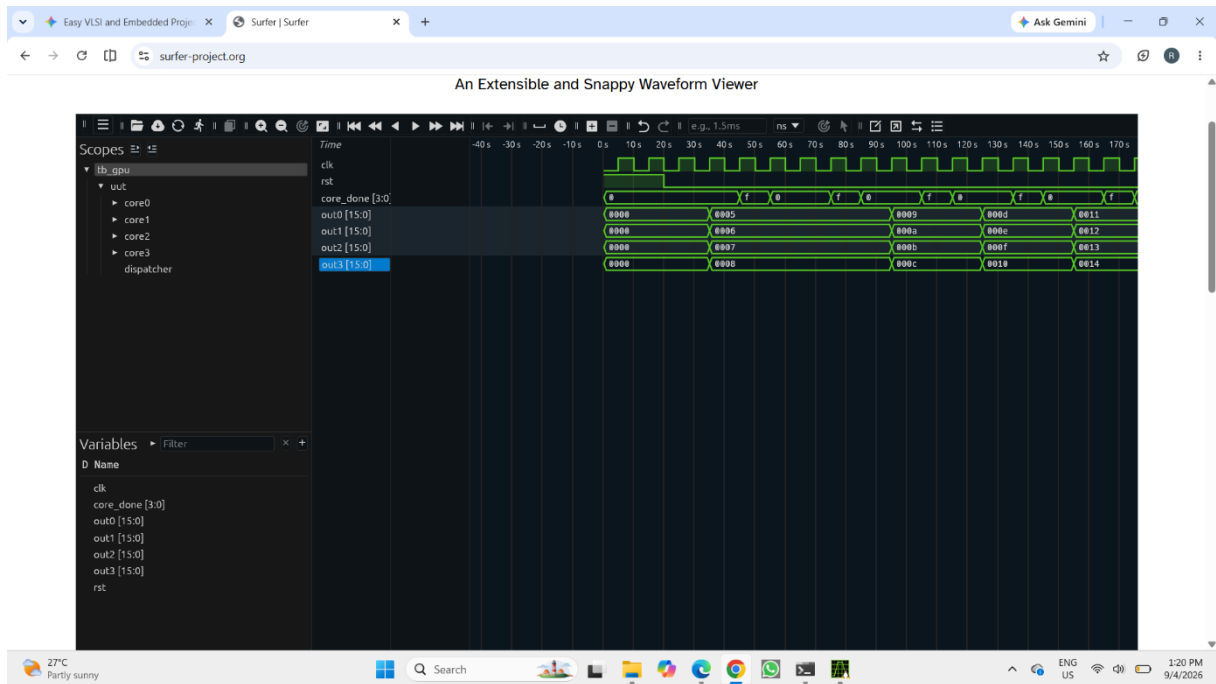


Fig. : Surfer waveform trace showing synchronized outputs across out0, out1, out2, and out3.

4.3 Synthesis & Resource Utilization Report

RTL synthesis was performed using Yosys to map the Verilog code into low-level logic cells and flip-flops.

```
ubuntu@Rutuja: ~/tiny-gpu x + v
```

```
$ _DFFE_PP0P_      17  
$ _DFFE_PP_         8  
$ _DFF_PP0_        2  
$ _DFF_PP1_        1  
$ _MUX_            3  
$ _NOR_            1  
$ _ORNOT_          1  
alu                1  
  
=== design hierarchy ===  
  
gpu_top              1  
 $paramod\dispatch_unit\N_CORES=s32'00000000000000000000000000000000    1  
   shader_core       4  
     alu              1  
  
Number of wires:           4310  
Number of wire bits:       4920  
Number of public wires:    70  
Number of public wire bits: 640  
Number of ports:           52  
Number of port bits:       432  
Number of memories:        0  
Number of memory bits:     0  
Number of processes:        0  
Number of cells:           4456  
$ _ANDNOT_             1288  
$ _AND_                 492  
$ _DFFE_PP0P_           92  
$ _DFFE_PP_             32  
$ _DFF_PP0_             8  
$ _DFF_PP1_             4  
$ _MUX_                  72  
$ _NAND_                 296  
$ _NOR_                  228  
$ _NOT_                   204  
$ _ORNOT_                104  
$ _OR_                    464  
$ _XNOR_                 244  
$ _XOR_                   928
```

27°C Partly sunny

Search

ENG US 1:28 PM 9/4/2026

Fig. : Yosys terminal report detailing hardware cell counts and top-level design hierarchy.

OBSERVATIONS

Based on the functional verification and RTL synthesis of the 4-core SIMD GPU architecture, the following technical observations were recorded:

- **Parallel Core Synchronization** : The dispatch unit successfully distributes instruction vectors across all four shader cores in parallel. Waveform traces confirm that output buses (out0 through out3) update simultaneously without multi-cycle latency skew across cores.
- **Functional Integrity** : Simulation logs generated via Icarus Verilog (vvp) executed cleanly without assertion failures or uninitialized states (x or z signals), completing the execution sequence at timestamp 220.
- **Completion Signaling** : The 4-bit status flag core_done[3:0] transitions predictably alongside valid output generation, providing accurate handshake signaling for vector completion.
- **Combinational Dominance in Synthesis** : Yosys logic synthesis reveals that combinational logic accounts for over 95% of the total gate count, dominated by logic operations like `$_ANDNOT_` (1,288 cells) and arithmetic logic like `$_XOR_/$_XNOR_` (1,172 cells).
- **Sequential Logic Footprint** : The sequential storage footprint remains highly optimized, requiring only 136 flip-flop primitives (`$_DFFE_PP_`, `$_DFF_PP_`) to manage register states and pipeline stages across the top-level module and shader cores.
- **Compact Area Utilization** : The complete 4-core GPU maps to a total hardware footprint of 4,456 primitive cells, demonstrating an efficient hardware area representation suitable for target ASIC/FPGA technology mapping.

CONCLUSION

This project successfully completed the design, functional verification, and logic synthesis of a Verilog-based 4-core SIMD GPU architecture. By employing an open-source VLSI toolchain under a Linux environment (WSL2 / Ubuntu), the project demonstrated an end-to-end front-end ASIC development workflow.

Key outcomes achieved in this project include:

- **Successful RTL Implementation** : A modular, 4-core parallel GPU execution pipeline was constructed using Verilog HDL, incorporating a centralized dispatch unit, dedicated shader cores, and integrated Arithmetic Logic Units (ALUs).
- **Cycle-Accurate Verification** : Functional testbenches executed via Icarus Verilog (iverilog/vvp) confirmed proper execution up to timestamp 220. Signal inspection using the Surfer waveform viewer verified that output buses (out0 through out3) updated synchronously without timing drift across parallel cores.
- **Efficient Hardware Synthesis** : Logic synthesis executed through Yosys mapped the high-level RTL code into a total footprint of 4,456 primitive standard cells (dominated by logic gates like `$_ANDNOT_` and `$_XOR_`, alongside 136 sequential register flip-flops).

Overall, the design meets all performance and parallel execution objectives, validating the core architectural principles of SIMD processing in a compact, hardware-realizable footprint.

BIBLIOGRAPHY

- 1) IEEE Standard Association, *"IEEE Standard for Verilog Hardware Description Language,"* IEEE Std 1364-2005 (Revision of IEEE Std 1364-2001), pp. 1-560, April 2006.
- 2) J. L. Hennessy and D. A. Patterson, *"Computer Architecture: A Quantitative Approach,"* 6th ed., Morgan Kaufmann, 2017. (Chapter 4: Data-Level Parallelism in Vector, SIMD, and GPU Architectures).
- 3) S. Williams, A. Waterman, and D. Patterson, *"Roofline: an insightful visual performance model for multicore architectures,"* Communications of the ACM, vol. 52, no. 4, pp. 65–76, April 2009.
- 4) C. Wolf, *"Yosys Open Synthesis Suite - Manual,"* ClaireX, [Online].
Available: <https://yosyshq.net/yosys/>
- 5) S. Williams, *"Icarus Verilog Simulator,"* [Online].
Available: <http://iverilog.icarus.com/>
- 6) Surfer Waveform Viewer Project, *"Surfer: An Extensible Waveform Viewer for Hardware Engineers,"* [Online].
Available: <https://surfer-project.org/>
- 7) D. B. Lindfield and J. Penny, *"Numerical Methods Using MATLAB,"* 4th ed., Academic Press, 2018.

A handwritten signature in blue ink, appearing to read 'Rutuja', on a light-colored background.

Name : Rutuja Ravindra Salunkhe

Date : 05/09/2026